

A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA

Tesina di Algoritmi e Strutture Dati
Corso di laurea in Ingegneria Informatica ed Elettronica
A.A. 2025/2026
Prof. E. Di Giacomo

Cache Manager

Implementazione di un gestore cache ottimo

Algoritmo Goloso Furthest-In-Future

A cura di:
Luis Alessandro Siancas Curi
Matricola 370778
luisalessandro.siancascuri@studenti.unipg.it

Sommario

1 - Introduzione e definizione del problema....3
1.1 Il contesto tecnologico: la gerarchia di memoria....3
1.2 Il problema della gestione della cache....3
1.3 Algoritmi per la gestione della cache....4
1.4 Obiettivi....5
2 - La logica dell'algoritmo OPT....6
2.1 La soluzione banale (Naive)....6
2.2 Creare la cache....7
2.3 Effettuare l'accesso ad una risorsa....9
2.4 Considerazioni sulla complessità....10
3 - Implementazione Software....11
3.1 L'albero rosso-nero e il max-heap....11
3.2 La tavola hash....13
3.3 Creazione degli algoritmi e analisi complessità....13
3.4 Interfaccia....15
4 - Validità dell'algoritmo....17
4.1 Proprietà della sottostruttura ottima....17
4.2 Proprietà della scelta golosa....18
5 - Prova sperimentale....20
5.1 Metodologia di esecuzione dei test....20
5.2 Prima prova con K=50 e M=200....20
5.3 Seconda prova per K crescente....23
5.4 Raccolta dei vari test....24
6 - Conclusioni e Sviluppi futuri....27
6.1 Conclusioni....27
6.2 Sviluppi Futuri....27

1 - Introduzione e definizione del problema

1.1 Il contesto tecnologico: la gerarchia di memoria

Nei moderni sistemi di elaborazione, le prestazioni complessive del sistema non dipendono unicamente dalla velocità di calcolo della CPU, ma sono fortemente influenzate dal **tempo** necessario per l'accesso ai dati. Per superare il divario prestazionale tra la velocità dei registri del processore e i tempi di risposta della memoria di massa (o della memoria RAM), l'architettura degli elaboratori adotta una **gerarchia di memoria**:

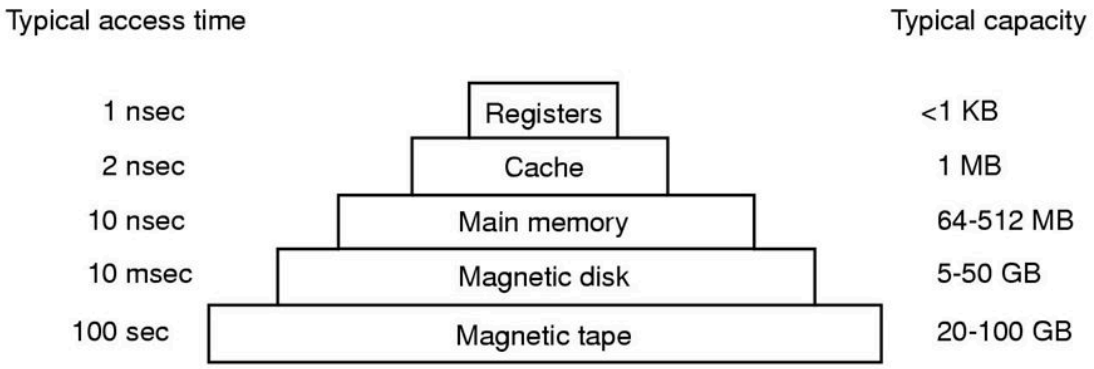


Figura 1.1: Gerarchia delle Memorie

In questa gerarchia, la memoria Cache si colloca nei livelli più alti: è una memoria estremamente veloce, ma a causa degli elevati costi di produzione e di vincoli fisici, ha una capacità fortemente limitata. Il principio cardine su cui si basa l'efficacia di questa architettura è il *principio di località (sia spaziale che temporale)*, secondo il quale un programma tende ad accedere frequentemente a un insieme ristretto di risorse in un breve periodo di tempo.

1.2 Il problema della gestione della cache

Il problema sorge quando un dato necessario alla CPU non è presente nella cache. In particolare, se essa è piena, è necessario prendere una risorsa presente nella cache e sostituirla con quella necessaria.
Volendo analizzare in maniera più rigorosa il fenomeno:

- **Esecuzione ordinaria:** la CPU è impegnata nell'esecuzione delle istruzioni di un processo in *modalità utente (user mode)*;

- **Verifica nei registri:** nel momento in cui è richiesta una risorsa non presente nei registri della CPU (il livello più alto, veloce e ridotto in quanto a capacità di memoria), il dato necessario viene cercato nei livelli della cache;
- **Cache miss e interruzione:** laddove la risorsa necessaria non dovesse trovarsi nella cache, si verifica il cosiddetto cache miss: tale evento scaturisce l'interruzione del processo in esecuzione (trap), memorizzando l'ultima riga di codice fino ad ora eseguita;
- **Cambio di contesto:** avviene dunque un *cambio di contesto (context switch)*: si entra in *modalità protetta (kernel mode)*, il controllo viene trasferito al sistema operativo e si procede a prelevare la risorsa necessaria;
- **Spazio non disponibile:** nel caso in cui la cache dovesse essere piena, è necessario scegliere quale delle risorse presenti sostituire con quella necessaria;

Di base l'obiettivo fondamentale di qualsiasi politica di gestione della cache è quello di **minimizzare il numero totale di cache miss** su una sequenza di richieste nel lungo periodo. Ridurre i miss significa infatti ridurre i cambi di contesti, che comportano accessi ai livelli inferiori della memoria (memoria centrale/secondaria), operazioni che richiedono chiaramente tempi di elaborazione ampiamente maggiori rispetto a quelli della cache.

Non avere una buona gestione della cache aumenta il rischio finire in un contesto detto *thrashing*: il tempo impiegato allo scambio dei dati supera quello dedicato all'esecuzione effettiva dei processi.

1.3 Algoritmi per la gestione della cache

In caso di cache miss, il Sistema Operativo deve quindi provvedere a recuperare la risorsa necessaria, effettuando eventuali sostituzioni.
Le soluzioni possibili prevedono due macro-categorie di algoritmi:

- **Algoritmi Online:** sono gli algoritmi utilizzati nei sistemi operativi reali. Questi algoritmi devono prendere una decisione di *sfratto (eviction)* in tempo reale, basandosi su informazioni attuali della cache, ed eventualmente anche sullo storico delle risorse che sono state allocate in passato;
- **Algoritmi Offline:** sono algoritmi puramente teorici che operano con la piena conoscenza della sequenza delle risorse che verranno richieste nel tempo. Vengono utilizzati come misura di paragone per valutare l'efficienza di un

algoritmo on-line. Non sono riproducibili nella realtà, in quanto ovviamente non è possibile conoscere le richieste future;

1.4 Obiettivi

Si vuole progettare un algoritmo offline, tale per cui in caso di cache miss e cache piena, l'algoritmo provveda a **rimuovere dalla cache la risorsa che verrà chiamata più lontano nel tempo** (futuro). Si assume nota la sequenza delle richieste.

Tale algoritmo è denominato **OPT (o Furthest-In-Future)**, e nonostante non sia implementabile nei sistemi operativi reali (per quanto spiegato prima), rappresenta il *limite inferiore (lower bound)* in quanto a cache miss possibili data una certa sequenza di richieste, ed è quindi un riferimento per il confronto con un qualunque algoritmo online, nel momento in cui bisogna analizzarne le sue prestazioni.

In particolare si studieranno:

- **Optimal:** la progettazione migliore per massimizzare le prestazioni temporali, la quale ha ottimi risultati grazie all'appoggio verso diverse strutture dati;
- **Naive (ingenuo):** una progettazione intuitiva in grado di ottenere comunque il numero minimo di cache miss possibile, ma con costi temporali decisamente maggiori;

Le due soluzioni verranno poste a confronto per marcare la differenza tra la progettazione intelligente contro la più intuitiva.

Inoltre si effettueranno dei test di confronto anche con alcuni algoritmi online:

- **FIFO (First-In-First-Out):** politica nella quale l'elemento da rimuovere è quello che tra le risorse presenti nella cache è stato inserito per primo;
- **LIFO (Last-In-First-Out):** politica nella quale l'elemento da rimuovere è quello che tra le risorse presenti nella cache è stato inserito per ultimo;

2 - La logica dell'algoritmo OPT

2.1 La soluzione banale (Naive)

Definiamo la procedura generica CACHE-MANAGER, che inizializza una cache tramite INITIALIZE-CACHE, e procede ad eseguire ACCESS per ciascuna risorsa richiesta.
L'ingresso a tale algoritmo consiste in un array $R=<r_1,r_2,...,r_n>$ contenente le n richieste, e ad un intero k che corrisponde alla dimensione della cache:

```
CACHE-MANAGER(R, k)
  INITIALIZE-CACHE(R, k)
  for i = 1 to n
      ACCESS(r_i)
```

Figura 2.1: pseudocodice della soluzione generica

La complessità asintotica di tale algoritmo è chiaramente dettata dalla complessità di ACCESS, la quale riceve in ingresso la richiesta i-esima: tutto dipende quindi da come progettiamo questa procedura.
Logicamente la prima cosa da fare è cercare la risorsa richiesta tra quelle presenti nella cache: in caso contrario, una prima idea potrebbe essere quella di scorrere tutti gli elementi in cache, e individuare per ciascuno la prossima chiamata nel futuro.
La risorsa da rimuovere non è altro che quella che verrà chiamata più lontano nel futuro.
Questo significa che in caso di cache miss e cache piena:

- devo scorrere ciascuna delle k risorse in cache;
- per ciascuna risorsa nella cache devo scorrere R a partire dalla richiesta che segue quella attuale;

Volendo calcolare la complessità di CACHE-MANAGER:

- il primo ciclo (Figura 2.1) prevede n iterazioni: $\Theta(n)$;
- all'interno di ACCESS devo scorrere k risorse: $\Theta(k)$;
- per ciascuna risorsa, nel caso peggiore devo scorrere tutte le richieste seguenti fino alla fine di R: $O(n)$;

Il che significa che la soluzione banale avrebbe complessità $\mathbf{O(kn^2)}$!
Cerchiamo quindi di capire come si potrebbe ricavare una soluzione migliore.

2.2 Creare la cache

```
INITIALIZE-CACHE(R, k)
  let T be a new red-black tree
  let P be a new max-priority queue
  let H be a new hash table
  ind = 1
  for i = 1 to n
      j = HASH-SEARCH(r_i)
      if j == NIL
          HASH-INSERT(r_i, ind)
          let S_ind be a new linked list
          j = ind
          ind = ind + 1
      append i to S_j
```

Figura 2.2: pseudocodice della creazione della cache

Innanzitutto per migliorare le prestazioni dell'algoritmo, è necessario appoggiarsi a delle strutture dati che faciliteranno l'accesso alle informazioni sulle risorse nella cache e sulle relative chiamate future. Si hanno:

- T: un albero rosso-nero i cui nodi sono le risorse presenti nella cache, dove ciascun nodo contiene come parametro key il nome della risorsa corrispondente. Le operazioni eseguibili sull'albero devono essere modificate in maniera tale da tenere conto del numero dei nodi che l'albero possiede: TREE-SIZE è la procedura che permette di restituire il numero dei nodi dell'albero in un tempo costante. Dato che l'albero possiede al limite k nodi (dimensione della cache), le operazioni di inserimento, cancellazione e ricerca hanno un costo $O(\lg k)$ nel caso peggiore;
- P: una coda di priorità implementata con max-heap che in concomitanza con l'albero memorizza le risorse presenti nella cache. Si suppone che ciascun nodo possieda due parametri:
    - key: contiene l'istante di tempo della prossima chiamata della risorsa corrispondente al nodo;
    - value: contiene il nome della risorsa;

  In questo modo la risorsa che verrà chiamata più lontano nel tempo corrisponderà alla radice dell'heap, il che significa che individuare il massimo avrà costo $\Theta(1)$ e la sua estrazione avrà costo $O(\lg k)$, così come l'inserimento di un nodo generico.

- H: una tabella hash che mappa le risorse in interi. L'associazione ad un intero ind permette di costruire una lista S_{ind} contenente gli istanti di tempo delle prossime chiamate di una certa risorsa r_i. La tabella deve essere implementata in maniera tale da potersi ingrandire dinamicamente nell'operazione di HASH-INSERT, mantenendo un fattore di carico α sotto una certa costante. In questo modo, il costo ammortizzato dell'inserimento è $\Theta(1)$, e supponendo valida l'ipotesi di hashing uniforme, il costo della ricerca è anch'esso $\Theta(1)$;

Una nota importante: T e P devono mantenere una relazione tra di loro per facilitare una serie di operazioni, dunque ciascun nodo in T ha un riferimento al corrispettivo nodo in P e viceversa. Le ragioni sono:

- **Cambio di priorità:** quando avviene un cache hit, la risorsa richiesta deve cambiare il parametro key in P per aggiornare il prossimo istante di chiamata. Il problema è che un max-heap è in grado di individuare solo la radice, mentre un nodo al suo interno lo costringerebbe a scorrere l'array nel quale sono allocati i suoi nodi, degradando le prestazioni a $O(k)$. Conoscendo però la risorsa r_i richiesta, la sua ricerca in T ha costo $\Theta(\lg k)$, e mantenendo un riferimento al corrispettivo nodo in P, il "salto" nell'heap è a costo costante! Tramite l'operazione INCREASE-KEY sarà quindi possibile modificare $r_i.key$ al prossimo istante, con una seguente fase di correzione dell'heap con un costo pari a $O(\lg k)$. Ne consegue che in caso di cache hit l'aggiornamento delle informazioni delle risorse nella cache ha costo $O(\lg k)$;
- **Sfratto di un nodo:** nel momento in cui avviene un cache miss e la cache è piena, è necessario rimuovere una risorsa in favore di quella richiesta. In questo caso P è in grado di individuare tale elemento in maniera istantanea (costo $\Theta(1)$), e la sua rimozione costerà $O(\lg k)$. Il problema è che T non ha modo di individuare quale sarà la risorsa che verrà richiesta più lontano nel futuro. Avere quindi un riferimento da P a T permette quindi di individuare immediatamente il nodo da rimuovere in T, e la rimozione nell'albero avrà costo $\Theta(\lg n)$. Gestire la cache in questa situazione ha quindi costo $O(\lg k)$.

Per quanto riguarda il resto dell'algoritmo, l'inizializzazione consiste nello scorrere R in maniera tale da popolare le liste mappate su ciascuna risorsa, dove il parametro ind scorre un indice per ciascuna lista S_{ind}.

2.3 Effettuare l'accesso ad una risorsa

```
ACCESS(r_i)
  // Compute the next access time for r_i.
  ind = HASH-SEARCH(r_i)
  time = ∞
  delete the head of S_ind
  if S_ind is not empty
      time = head of S_ind
  // Check to see whether r_i is currently in the cache.
  if TREE-SEARCH(T.root, r_i) ≠ NIL
      print "cache hit"
  elseif TREE-SIZE(T) < k
      // Insert in an empty slot in the cache.
      let z be a new node for T
      z.key = r_i
      RB-INSERT(T, z)
      let event be a new object for P
      event.key = time
      event.value = r_i
      INSERT(P, event)
      print "cache miss, inserted " r_i " in empty slot"
  else event = MAXIMUM(P)
      if event.key ≤ time          // r_i has the furthest access time
          print "cache miss, no data element evicted"
      else // evict the element with furthest access time
          print "cache miss, evict data element " event.value
          event = EXTRACT-MAX(P)
          RB-DELETE(T, TREE-SEARCH(T.root, event.value))
          event.key = time
          event.value = r_i
          INSERT(P, event)
          let z be a new node for T
          z.key = r_i
          RB-INSERT(T, z)
```

Figura 2.3: pseudocodice dell'accesso a una risorsa

Per quanto riguarda l'accesso ad una risorsa:

- Si cerca l'indice della lista associata, si fissa l'istante di tempo della prossima chiamata (se l'attuale richiesta è l'ultima, si avrà time=∞);
- Si verifica la presenza di tale risorsa nella cache;
- In caso contrario, si verifica che ci sia spazio nella cache, ed in caso positivo, si alloca la risorsa;

- Se la cache è piena, si osserva se l'istante di tempo della prossima chiamata di r_i è maggiore del massimo dell'heap: in tal caso nonostante il cache miss, non avverrà alcuna rimozione/allocazione nella cache;
- Altrimenti, si procede alla rimozione della risorsa nella radice dell'heap, con analoga rimozione nell'albero rosso-nero, e si conclude allocando la risorsa richiesta in entrambe le strutture;

Volendo studiare la complessità di tale procedura:

- Nel caso di cache hit o di cache miss senza alcuna rimozione, il costo di tali operazioni è costante;
- I **casi dominanti** avvengono quando in caso di cache miss la cache ha ancora spazio disponibile, oppure quando è necessaria la rimozione di un elemento in favore della risorsa richiesta. Tali casi infatti prevedono inserimenti e/o cancellazioni sia nell'heap che nell'albero rosso nero, con costo logaritmico rispetto al numero dei nodi delle due strutture;

Per definizione, le due strutture possiedono al limite k nodi, dove k è la dimensione della cache.
Ne consegue che nel caso peggiore, la procedura ACCESS ha costo $O(\lg k)$.

2.4 Considerazioni sulla complessità

- La fase di inizializzazione, scorre tutto R e comprende operazioni di ricerca e di inserimento nella tabella H, che visto le considerazioni precedenti sono a costo $\Theta(1)$. Viste le n iterazioni per lo scorrimento di R, si ha: $\Theta(n)$;
- Segue lo scorrimento delle n richieste di R, il che prevede n iterazioni, dove ciascuna esegue la procedura ACCESS per la risorsa richiesta;
- Sappiamo che per quanto detto prima, ACCESS ha complessità $O(\lg k)$;

Ne consegue che la procedura CACHE-MANAGER ha complessità $\mathbf{O(n*\lg k)}$, un costo decisamente inferiore a quello calcolato per la soluzione banale, che ricordiamo avere complessità $\mathbf{O(kn^2)}$.

3 - Implementazione Software

Si è deciso di implementare gli algoritmi sopra mostrati in ambiente Java, nel seguente modo:

- una classe per ciascun algoritmo, con uno stampo generico dove il costruttore inizializza la cache ed un metodo ACCESS con la relativa politica in base all'algoritmo implementato. Si include poi un metodo per ricavare il numero dei cache miss;
- classi dedite alle strutture dati di albero rosso-nero e max-heap, in seguito si illustrerà meglio il perché di questa scelta;
- una classe di supporto nella lettura di file .txt per la conversione in un array R, creazione di sequenze casuali e produzione di file .csv per l'analisi delle performance di ciascun algoritmo;
- una classe per la simulazione degli algoritmi, dati certi input, con misurazione di cache miss e tempi di esecuzione;

3.1 L'albero rosso-nero e il max-heap

Un albero rosso-nero è un'implementazione degli alberi binari di ricerca, la quale consente (a differenza degli abr) di mantenere l'altezza dell'albero logaritmica rispetto al numero dei nodi che lo compongono.
In particolare, per un albero rosso-nero devono valere le seguenti proprietà:

- un nodo è rosso o nero;
- la radice è nera;
- le foglie (cioè la sentinella nil) sono nere;
- se un nodo è rosso, entrambi i suoi figli sono neri;
- per ogni nodo, tutti i percorsi che partono da tale nodo e vanno verso i suoi discendenti contengono lo stesso numero di nodi neri;

Un max-heap è una coda di priorità che consiste in un albero binario quasi completo la cui caratteristica è che per ciascun nodo, i sottoalberi radicati nei suoi figli contengono nodi le cui chiavi sono minori o uguali rispetto alla chiave di tale nodo. Ne consegue che la radice contiene la chiave massima tra quelle di tutti i nodi.

A tal proposito, le librerie standard di Java forniscono le classi Treemap e PriorityQueue. Tuttavia si è deciso di ricostruire le due strutture, date le seguenti necessità:

- **Legame tra le due strutture:** le due classi sono indipendenti e non si avrebbe modo di legare i puntatori della coda a quelli dell'albero e viceversa;
- **Cambio di priorità:** cambiare la chiave di un nodo interno ad una PriorityQueue richiede l'esecuzione delle operazioni remove(Object) e contains(Object), con complessità O(k) data la ricerca lineare dei nodi. Come segnalato in precedenza infatti, l'accesso ad un nodo per una modifica alla chiave parte dall'albero, che appunto non è in grado di comunicare con questa classe;
- **Nodi personalizzati:** riprendendo i due punti precedenti, al fine di ottimizzare le operazioni nelle due strutture, è necessario che ciascun nodo dell'albero abbia un riferimento al corrispondente nodo nella coda e viceversa. Tale riferimento permetterà un accesso ad una struttura partendo dall'altra, il tutto a costo costante;

Per quanto riguarda l'**albero rosso-nero, esso è stato completamente riprodotto** da zero, si sono incluse le procedure di ricerca, inserimento e cancellazione, accompagnate dalle dovute operazioni di supporto, come la transplant, le rotazioni, il minimo di un sottoalbero, e chiaramente le fixup necessarie a correggere le violazioni causate da inserimento e cancellazione.
Si sono escluse operazioni come quelle per il calcolo del successore/predecessore, oppure per la ricerca del massimo di un sottoalbero, in quanto non strettamente necessarie per lo scopo del progetto.
Si hanno due classi:

- RBNode: una classe estremamente semplice, volta a rappresentare un singolo nodo di un albero rosso-nero. Ciascun oggetto RBNode possiede tra gli attributi un oggetto HeapNode, un riferimento al rispettivo nodo nella coda;
- RBTree: la classe che implementa l'albero rosso-nero, i cui nodi sono oggetti RBNode;

Il **max-heap è stato ricostruito tramite un ArrayList**, un'implementazione dell'interfaccia List, che fa da wrapper degli array: il vero vantaggio sta nel fatto che a differenza di un array standard, in questo caso non è richiesta la dichiarazione dello spazio necessario, e proprio come le liste, è una struttura che cresce dinamicamente.

Dato che per definizione un max-heap è un albero binario quasi completo, l'implementazione tramite array permette un accesso a figli/genitore di un nodo in maniera molto rapida. Si possono infatti definire:

- PARENT(i) → return $\lfloor i/2 \rfloor$;
- LEFT(i) → return 2*i;
- RIGHT(i) → return 2*i+1;

Tali procedure operano sugli indici di un array, il che spiega la comodità di appoggiarsi a tale struttura (nell'implementazione in Java, visto che gli indici partono da 0 e non da 1 come negli pseudocodici, tali operazioni vengono ridefinite di conseguenza). Inoltre la ArrayList possiede il metodo size(), utile nel momento di estrazione del massimo (scambio radice con ultimo nodo), il che evita la creazione del parametro heap-size, necessario nell'implementazione dei max-heap.

3.2 La tavola hash

Una tavola hash è un array di dimensione m, il quale si appoggia ad una funzione hash che mappa le chiavi in indici dell'array. La funzione deve essere progettata in maniera tale da avere come codominio tutte le posizioni della tavola, dunque <0,1,...m-1>.
Il problema da gestire è dato dal numero di elementi da allocare rispetto alle posizioni disponibili. Definito n come il numero di elementi da allora ed m come le posizioni disponibili nella tavola, si definisce il fattore di carico $\alpha=n/m$. Come descritto all'inizio, la tavola deve essere tale da prevedere una tecnica di ingrandimento al fine di mantenere α sotto una certa costante: tutto ciò è necessario per non far crescere troppo la probabilità di conflitti.

In questo caso si è usata una **HashMap<String,LinkedList<Integer>>**, una struttura già presente nelle librerie di Java che mappa una stringa (la risorsa), associandola direttamente ad una lista doppiamente collegata i cui nodi contengono interi (gli istanti di tempo delle prossime chiamate di una certa risorsa).

- HashMap garantisce costo ammortizzato $O(1)$ per le operazioni di ricerca e inserimento;
- LinkedList garantisce costo $O(1)$ nell'estrazione della testa;

3.3 Creazione degli algoritmi e analisi complessità

Per questo progetto, l'implementazione dell'algoritmo OPT è l'unica che richiede come appoggio le strutture dati menzionate nei paragrafi precedenti. Per quanto riguarda la soluzione banale, essa si appoggia su un array di dimensione k che

rappresenta la cache: la logica banale e il non appoggio a strutture dati ideali per il contesto sono la causa della sua elevata complessità rispetto alla soluzione ottima.

Anche le implementazioni degli algoritmi FIFO e LIFO non sono particolarmente elaborate. Entrambe si basano su un array di dimensione k analogo a quello sfruttato per la soluzione banale (Naive).
Riguardo alla politica di gestione della cache:

- FIFO usa un indice $evicted$ per puntare all'elemento presente in cache da più tempo. Tale indice è quindi utile nel momento della sostituzione di tale risorsa con una richiesta, e si aggiorna circolarmente;
- LIFO sfrutta il parametro k per individuare l'elemento da rimuovere e sostituire. Questo perché la cache viene popolata dalla posizione 0 fino alle $k\text{-}1$, e visto che l'ultimo elemento aggiunto si trova in posizione $k\text{-}1$, è chiaro che tale elemento verrà rimosso in caso di richiesta di una risorsa non presente in cache;

Per quanto riguarda le complessità degli algoritmi:

- **Optimal:** $O(n*\lg k)$;
- **Naive:** $O(kn^2)$;
- **FIFO:** la fase più dispendiosa corrisponde allo scorrimento della cache per verificare la presenza della risorsa richiesta. Se si possiede un puntatore all'elemento più "vecchio", la rimozione di un elemento in favore di un altro è a costo costante. Dato che vanno gestite n richieste (dimensione di R), risulta che FIFO ha costo $O(kn)$;
- **LIFO:** valgono le considerazioni fatte per FIFO. Dunque la complessità dell'algoritmo è dettata dallo scorrimento della cache volto a verificare la disponibilità della risorsa richiesta. In caso negativo, che sia presente spazio o meno, l'inserimento della risorsa richiesta con eventuale rimozione di un'altra risorsa, è a costo costante. E visto che l'algoritmo prevede la gestione di n richieste, anche in questo caso il costo è $O(kn)$;

3.4 Interfaccia

Non si è progettata una vera e propria interfaccia grafica: si tratta di un menù stampato su console che permette di scegliere tra due modalità:

1. **Presentazione:** si esegue un test su dei file di testo già preparati, così da poter confrontare i risultati con quelli dell'analisi sperimentale (capitolo 5);
2. **Live:** l'utente può decidere la lunghezza della sequenza, la dimensione della cache ed il numero delle risorse (distinte). A quel punto il programma procede a creare un file casuale che rispetti le specifiche fornite dall'utente ed esegue tutti gli algoritmi con l'input costruito;

```
================================================
                SIMULATORE DI CACHE
================================================
1) Modalità Presentazione
2) Modalità Live
Scegli un'opzione (1 o 2):
```

Figura 3.1: menù principale del simulatore su console

```
> ESECUZIONE FILE PRE-CONFIGURATI

> Dimensione Cache (K): 100
> Numero risorse distinte (M): 400

File Elaborato       N          T_Optimal(ms)   T_Naive(ms)     T_FIFO(ms)      T_LIFO(ms)
-------------------------------------------------------------------------------------------
n5000_m200.txt       5000       11,25           57,16           2,59            1,96
n25000_m200.txt      25000      9,45            126,47          10,23           7,96
n50000_m200.txt      50000      9,77            241,44          16,11           6,75
n250000_m200.txt     250000     38,75           1168,21         48,57           33,35
n500000_m200.txt     500000     117,57          2268,35         124,67          65,59
-------------------------------------------------------------------------------------------
Dati della simulazione presenti in: output/risultati_benchmark.csv
```

Figura 3.2: esempio di esecuzione in modalità Presentazione

```
> ESECUZIONE PER FILE GENERATO RANDOMICAMENTE

1. Inserisci il nome del file da creare (es: test_live): test
2. Quante richieste totali deve contenere la sequenza (N)? 1000
3. Quale deve essere la dimensione della cache (K)? 10
4. Quante risorse distinte devono esserci nel test? 40

> Generazione del file casuale in corso...
> File creato con successo in: input/test.txt
> Caricamento del file e avvio simulazione...

================================================
           RISULTATI DELLA SIMULAZIONE
================================================
> Risorse totali elaborate (N): 1000
> Dimensione Cache (K): 10
> Numero di risorse distinte (M): 40
Algoritmo       Cache Miss      Tempo CPU (ms)
------------------------------------------------
Optimal         450             4,513
Naive           450             3,785
FIFO            755             0,322
LIFO            763             0,448
------------------------------------------------
Dati della simulazione inseriti in: test.csv
```

Figura 3.3: esempio di esecuzione in modalità Live

Qualunque modalità venga scelta, il test eseguito riporterà i risultati in quanto a cache miss e tempi di esecuzione di ciascun algoritmo in un file .csv dedicato, che si potrà poi caricare in un foglio di calcolo per valutare meglio i dati ottenuti.

4 - Validità dell'algoritmo

L'algoritmo che stiamo studiando è un algoritmo goloso, dove la scelta golosa consiste nel rimuovere la risorsa che verrà richiamata più lontano nel tempo.
E' quindi necessario dimostrare la proprietà della sottostruttura ottima e la proprietà della scelta golosa, affinché la risoluzione tramite algoritmo goloso sia valida.

4.1 Proprietà della sottostruttura ottima

Dimostrare questa proprietà significa che una **soluzione ottima al problema contiene soluzioni ottime ai suoi sotto-problemi**.

Si definisce il generico sottoproblema (C,i), dove:

- C è lo stato (il contenuto) della cache prima di servire la richiesta i-esima;
- i è l'indice della richiesta corrente in R;

La soluzione per (C,i) è l'**insieme di tutte le decisioni** che vengono realizzate dalla richiesta i-esima fino alla n-esima. Si definisce S come soluzione ottima al sottoproblema (C,i).

Supponiamo di trovarci quindi alla richiesta i-esima, si hanno due scenari:

- Cache hit: in questo caso la risorsa è presente nella cache, quindi il suo contenuto non cambia e il prossimo sottoproblema da risolvere è $(C,i+1)$;
- Cache miss: in questo caso è necessario allocare la risorsa necessaria, dunque lo stato della cache passa da C a C' (tramite aggiunta in uno slot libero o rimozione di un'altra risorsa), il nuovo sottoproblema da risolvere è $(C', i+1)$;

Sia S la soluzione ottima per $C(i,j)$ e sia S' la soluzione per il sottoproblema per $i+1$: si vuole dimostrare che S' è anch'essa ottima per il sottoproblema generato.
Supponiamo per assurdo che S' non sia soluzione ottima del sottoproblema per $i+1$, sia allora S'' soluzione ottima per tale sottoproblema. Posso quindi ricostruirmi una soluzione per $C(i,j)$ combinando la scelta fatta alla richiesta i-esima e la soluzione S''. La nuova soluzione costruita avrebbe però un numero totale di cache miss inferiore a quelli di S, che per ipotesi si era assunta come soluzione ottima per il problema.

Supponiamo di avere cache miss alla richiesta i-esima. Definiamo P_C come i possibili stati della cache a seguito di una decisione presa alla richiesta i-esima: P_C contiene $k+1$ elementi, dove k sono le possibili nuove combinazioni sostituendo una risorsa con quella richiesta, mentre il $+1$ corrisponde al caso in cui nonostante il cache miss non si preveda alcuna rimozione vista la prossima chiamata di r_i.

Denoto con $\mathrm{miss}(C,i)$ il numero minimo di cache miss per (C,i). Risulta:

$$miss(C,i) = \begin{cases} 0 & \text{if } i = n \text{ and } r_n \in C \ , \\ 1 & \text{if } i = n \text{ and } r_n \notin C \ , \\ miss(C, i+1) & \text{if } i < n \text{ and } r_i \in C \ , \\ 1 + \min\limits_{C' \in P_C} \{miss(C', i+1)\} & \text{if } i < n \text{ and } r_i \notin C \ . \end{cases}$$

Figura 4.1: ricorrenza di miss(C,i)

4.2 Proprietà della scelta golosa

Dimostrare questa proprietà significa che **la scelta golosa fa sempre parte di una soluzione ottima** al problema.

Oltre alle definizioni precedentemente illustrate, si ha:

- C_{Ai}: lo stato della cache per la soluzione A prima della richiesta i-esima;

Teorema: Sia A una soluzione ottima per (C,i). Sia $b \in C_{Ai} \cup \{r_i\}$ la risorsa la cui prossima richiesta avverrà più lontano nel tempo in assoluto (a partire dall'istante i-esimo), precisamente all'istante futuro m.
Allora è possibile costruire una soluzione alternativa A' per (C,i) che gode di queste proprietà fondamentali:

1. Alla richiesta i-esima, rimuove b;
2. Per tutti i passi intermedi tra $i+1 \leq j \leq m$, il contenuto delle cache di A e A' differisce al massimo per un solo elemento: b che si trova in C_{Aj} e non in $C_{A'j}$. In particolare, si possono denotare $C_{Aj}=D_j \cup \{b\}$ e $C_{A'j}=D_j \cup \{x\}$, con $x \neq b$;
3. In tutti questi passi intermedi, ogni volta che A registra un cache hit, anche A' registra un cache hit;
4. $C_{Aj}=C_{A'j}$ per $j>m$;
5. Il numero totale di miss prodotti da A' non supera mai quello di A;

Prova: se alla richiesta i-esima A rimuove b, allora la dimostrazione è ovvia ($A=A'$). Valutiamo il caso in cui A rimuova a, tale che $a \neq b$. Costruiamo una nuova soluzione A' che comprenda la scelta golosa:

- $A'=A \setminus \{a\} \cup \{b\}$, le due soluzioni differiscono per un solo elemento;
- Procediamo per induzione partendo dalla proprietà 1, per la quale si ha che $C_{A,i+1}$ e $C_{A',i+1}$ differiscono per un solo elemento. Se A esegue rimozioni che coinvolgono elementi in D_j, facciamo in modo che A' esegua le stesse rimozioni. Per tutto l'intervallo $i+1 \leq j < m$ siamo certi che b non verrà mai

richiesta, dato che per ipotesi essa è la risorsa che verrà chiamata all'istante m. All'istante $j=m$, A procede a rimuovere b, allora A' rimuoverà a;

- Se A ha un cache hit all'istante j, per $i+1 \leq j < m$, allora $r_j \in D_j$, visto che b verrà richiesta all'istante j. Visto che D_j è l'insieme comune tra A e A', $r_j \in C_{A'j}$, quindi anche A' avrà un cache hit;
- Grazie alla proprietà 2, si ha che C_{Am} differisce da $C_{A'm}$ al limite per un elemento (ossia b).
    - Se $C_{Am}=C_{A'm}$, allora all'istante m la decisione presa per A sarà ripetuta anche in A';
    - Se $C_{Am} \neq C_{A'm}$, allora all'istante m dato che $b \in C_{Am}$, A avrà un cache hit e A' un cache miss. Per A' quindi si applica la rimozione di a;

  Da questo punto in poi, ossia dall'istante $m+1$ in poi, A' esegue le stesse decisioni di A;
- Per la proprietà 3, nei passi intermedi tra $i+1$ e m, qualunque cache hit registrato in A viene registrato anche in A'. Dimostriamo che A e A' a questo punto hanno lo stesso numero di cache miss:
    - se all'istante m A registra un cache miss, allora la dimostrazione è fatta (poiché anche A' registra un cache miss, ma per b);
    - altrimenti bisogna dimostrare che A registra un cache miss per un certo istante j, tale che $i+1 \leq j < m$, istante nel quale A' registra un cache hit. Siccome A all'istante i rimuove a, allora per costruzione con A', si ha $C_{A',i+1}=D_{j+1} \cup \{a\}$. Chiaramente se in $i+1 \leq j < m$ A ha un cache hit, esso sicuramente non sarà per b, dato che questo implicherebbe un cache miss per A', con conseguente allocazione di b (eppure b è la risorsa che verrà chiamata per $j=m$). L'unico cache hit per A verso b, avverrà quindi all'istante m. Per la costruzione di A', siamo certi che a non viene mai rimosso. E sicuramente a è una richiesta che avviene a un certo istante j per $i+1 \leq j < m$, quindi in tale istante A registrerà cache miss;

In conclusione **possiamo costruirci una soluzione ottima con il minimo numero di cache miss contenente la scelta ottima**, dunque la proprietà risulta dimostrata.

5 - Prova sperimentale

5.1 Metodologia di esecuzione dei test

Per verificare l'efficienza reale degli algoritmi e confermare le complessità teoriche studiate nel Capitolo 2, sono state condotte una serie di simulazioni in ambiente **Java Virtual Machine (JVM)**.
Al fine di azzerare il **"rumore statistico"** dovuto ai processi in background del Sistema Operativo e compensare la fase di warm-up iniziale del compilatore Java, ogni singolo test è stato eseguito consecutivamente per **15 volte**. Il tempo di calcolo definitivo assegnato a ciascun algoritmo è dato dalla media aritmetica di queste esecuzioni, garantendo così una misurazione stabile e accurata.

Nelle seguenti prove K corrisponde alla dimensione della cache, R al numero di richieste e M al numero di risorse distinte.
Il test misurano, per ciascun algoritmo, il **numero di cache miss totale** ed il **tempo di esecuzione (medio)**, misurato tramite System.nanoTime() e convertito poi in ms.

5.2 Prima prova con K=50 e M=200

Valutiamo cosa accade al crescere del numero delle richieste, fissando K ed M:

N	K	Miss Optimal	Miss Naive	Miss FIFO	Miss LIFO	Time Optimal	Time Naive	Time FIFO	Time LIFO
5000	50	2085	2085	3767	3751	13,441	48,739	1,808	1,487
25000	50	10195	10195	18738	18870	9,012	112,949	6,930	5,942
50000	50	20323	20323	37628	37597	10,035	166,279	7,121	3,902
250000	50	101318	101318	187327	187366	42,064	858,486	41,069	18,879
500000	50	202563	202563	374997	374861	90,853	1828,081	81,239	40,326

Figura 5.1: tabella dei risultati di 5 test per N crescente, K=50, M=200

Al crescere del numero delle richieste notiamo immediatamente le differenze di esecuzione tra la soluzione banale e quella ottimale, il che potenzialmente rispecchierebbe le analisi fatte inizialmente.

E' interessante osservare poi come gli algoritmi online vincano a livello temporale contro la soluzione ottimale: il tutto è motivato dal fatto che le due politiche operano in maniera molto più semplice, senza dover guardare in avanti (ma neanche all'indietro). Il **vero prezzo da pagare per queste due soluzioni è dato dal numero di**

cache miss che conteggiano, che al crescere del numero delle richieste si allontana sempre di più dal limite inferiore calcolato dagli algoritmi offline.
Oltretutto in questi test non si è considerato il fatto che un cache miss comporti un cambio di contesto, una situazione che richiede tempi di elaborazione elevati visto l'accesso a livelli di memoria inferiori. A quel punto, siamo certi che FIFO e LIFO avrebbero tempi di esecuzione decisamente superiori a quelli di Optimal.

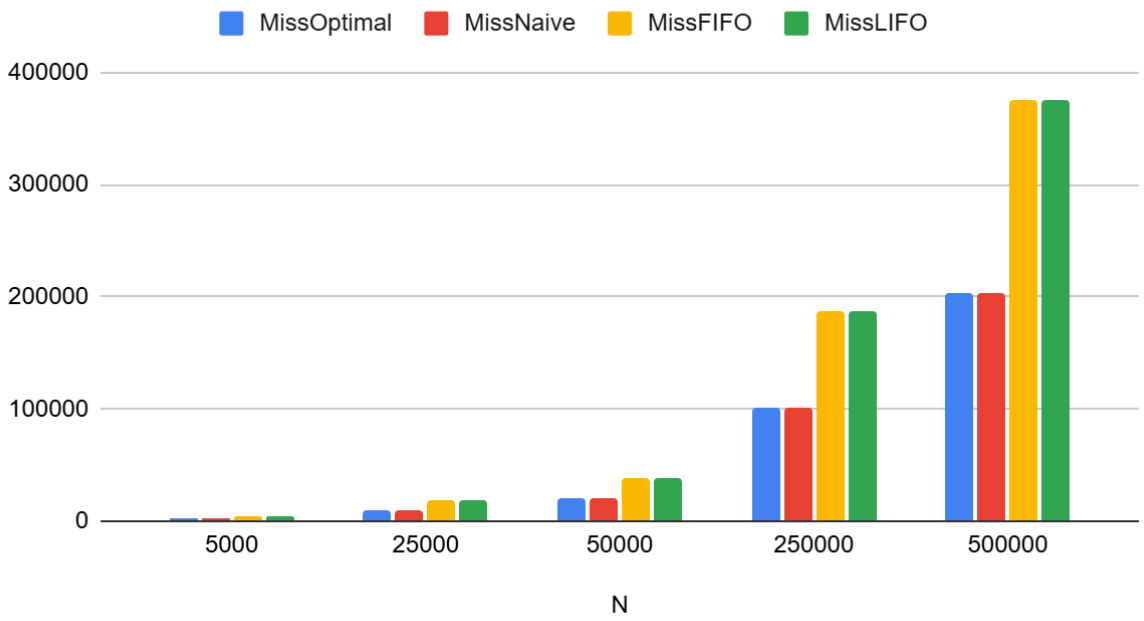


Figura 5.2: grafico a colonne del numero di cache miss per N crescente, K=50, M=200

Il grafico soprastante denota in maniera più chiara come il vero vantaggio dell'algoritmo Optimal stia nel ridurre al minimo i cache miss, e dunque i cambi di contesto.

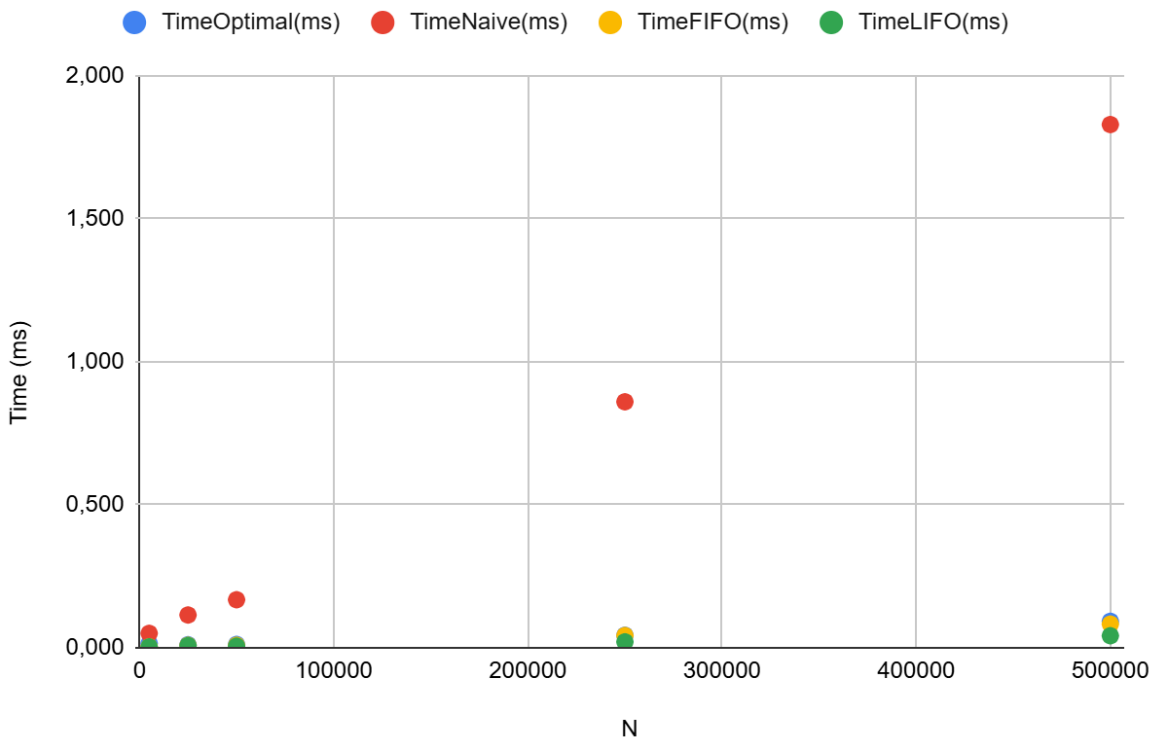


Figura 5.3: grafico dei tempi di esecuzione per N crescente, K=50, M=200

Se paragonati a Naive, gli altri algoritmi studiati hanno tempi di esecuzione pressoché paragonabili, basti vedere come in larga scala i punti di ciascun algoritmo si sovrappongano quasi.
E' evidente l'andamento lineare di FIFO e LIFO, dato che fissata K, dipendono esclusivamente da N.
Anche Optimal ha di base un andamento lineare: fissata K, $\lg(K)$ restituisce sempre un valore costante, il quale viene moltiplicato per N, variabile indipendente.
Per quanto riguarda Naive, probabilmente qualche ulteriore test permetterebbe di riconoscere meglio l'andamento quadratico, ma è comunque facile vedere come la crescita temporale di Naive sia decisamente superiore a quella degli altri algoritmi.

5.3 Seconda prova per K crescente

Nella prova precedente si era assunta K costante. Proviamo a vedere cosa accade fissando N e aumentando lo spazio della cache:

N	K	Miss Optimal	Miss Naive	Miss FIFO	Miss LIFO	Time Optimal	Time Naive	Time FIFO	Time LIFO
50000	50	20323	20323	37628	37597	10,035	166,279	7,121	3,902
50000	100	10063	10063	24996	25083	8,425	203,443	9,538	5,789
500000	50	202563	202563	374997	374861	90,853	1828,081	81,239	40,326
500000	100	100038	100038	249792	249817	75,343	2025,46	91,533	61,975

Figura 5.4: prova per K crescente e due valori fissati di N, M=200

Notare come tutti gli algoritmi degradano in quanto a tempi di esecuzione, meno che Optimal: avendo una cache più grande, l'algoritmo è in grado di minimizzare ulteriormente i cache miss, e conseguentemente, diminuiscono i contesti in cui è necessario rimuovere/aggiungere nodi alle strutture dati di supporto.
Per quanto riguarda la soluzione banale, il problema risiede sempre nella sua semplicità: pur ingrandendo la cache, il cardine del costo temporale risiede nella logica di ricerca e sostituzione dei nodi, visto che Naive non si appoggia ad alcuna struttura e procede a risolvere sottoproblemi che si ripetono nel tempo.

Logicamente i cache miss diminuiranno per tutti gli algoritmi:

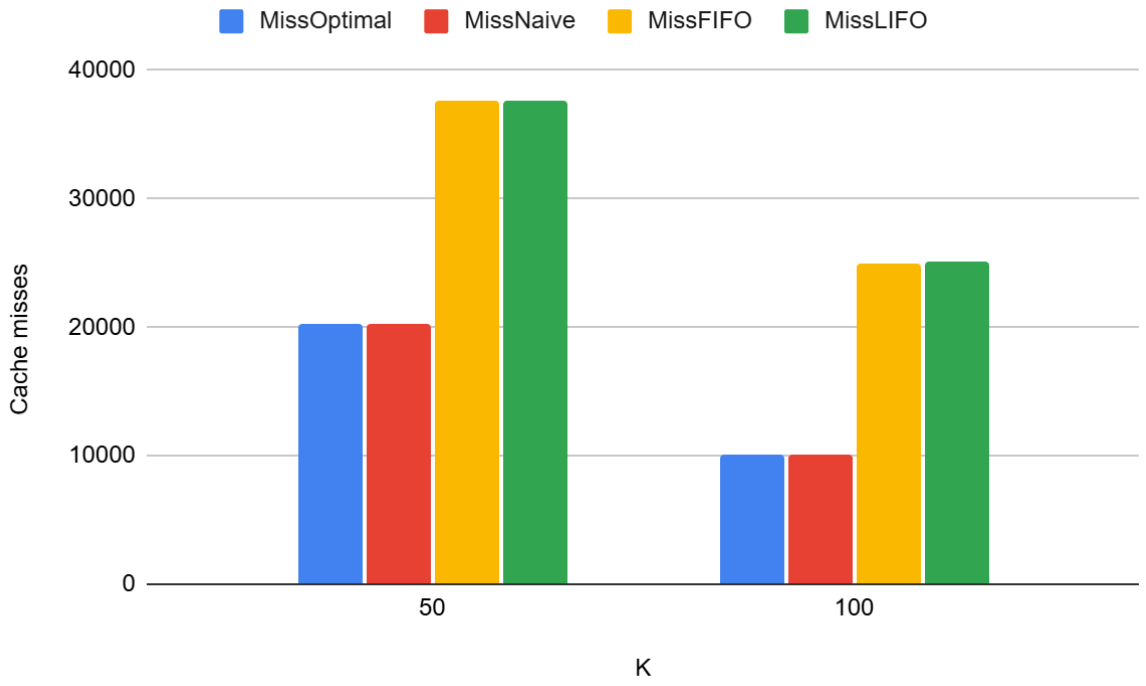


Figura 5.5: numero di cache miss per K=50 e K=100

5.4 Raccolta dei vari test

Nei paragrafi precedenti si è illustrata una porzione dei test effettuati al fine di denotare cosa accade al variare dei parametri di ingresso di ciascun algoritmo: le richieste (array R con N elementi) e la dimensione della cache (K).

Di seguito i risultati di una serie di test fatti per:

- K=50, 100;
- N=5000, 25000, 50000, 250000, 500000;
- M=200, 400;

N	K	Miss Optimal	Miss Naive	Miss FIFO	Miss LIFO	Time Optimal	Time Naive	Time FIFO	Time LIFO
5000	50	2085	2085	3767	3751	13,441	48,739	1,808	1,487
25000	50	10195	10195	18738	18870	9,012	112,949	6,930	5,942
50000	50	20323	20323	37628	37597	10,035	166,279	7,121	3,902
250000	50	101318	101318	187327	187366	42,064	858,486	41,069	18,879
500000	50	202563	202563	374997	374861	90,853	1828,081	81,239	40,326

Figura 5.6: risultati per K=50, M=100, N crescente

N	K	Miss Optimal	Miss Naive	Miss FIFO	Miss LIFO	Time Optimal	Time Naive	Time FIFO	Time LIFO
5000	50	2877	2877	4393	4353	10,802	78,299	1,866	1,558
25000	50	14166	14166	21887	21879	8,049	154,432	7,235	6,094
50000	50	28196	28196	43656	43797	11,420	272,246	6,133	3,770
250000	50	141066	141066	218648	218600	47,661	1385,992	33,431	17,977
500000	50	282564	282564	437630	437108	86,112	2766,850	59,706	32,370

Figura 5.7: risultati per K=50, M=200, N crescente

N	K	Miss Optimal	Miss Naive	Miss FIFO	Miss LIFO	Time Optimal	Time Naive	Time FIFO	Time LIFO
5000	100	1099	1099	2531	2502	11,682	53,801	2,488	2,156
25000	100	5103	5103	12547	12619	8,245	117,259	8,611	7,406
50000	100	10063	10063	24996	25083	8,425	203,443	9,538	5,789
250000	100	49975	49975	124825	125033	39,764	1025,272	46,764	31,507
500000	100	100038	100038	249792	249817	75,343	2025,460	91,533	61,975

Figura 5.8: risultati per K=100, M=100, N crescente

N	K	Miss Optimal	Miss Naive	Miss FIFO	Miss LIFO	Time Optimal	Time Naive	Time FIFO	Time LIFO
5000	100	2078	2078	3761	3735	11,969	92,662	2,979	2,804
25000	100	10000	10000	18806	18866	9,108	268,324	8,918	7,825
50000	100	19939	19939	37424	37624	10,265	487,688	9,254	6,200
250000	100	99443	99443	187347	187370	50,451	2466,248	45,573	36,246
500000	100	199103	199103	375003	374727	95,411	4991,461	90,602	60,750

Figura 5.9: risultati per K=100, M=200, N crescente

Ricapitolando ed aggiungendo delle ultime considerazioni:

- Fissato K, al crescere di N aumentano i tempi di esecuzione di ciascun algoritmo, ma in particolare dell'algoritmo ingenuo (Naive): tutto ciò è causato dal fatto che l'algoritmo non si appoggia a strutture dati elaborate e provvede a risolvere di volta in volta problemi che si ripresenteranno anche in futuro;
- Fissato N, al crescere di K, tutti gli algoritmi meno Optimal, nonostante registrino meno cache miss, impiegano più tempo nella loro esecuzione, il che dimostra quanto prestazionale sia Furthest-In-Future (Optimal). In particolare in questo modo è evidente come usare strutture dati d'appoggio e un approccio goloso forniscano una soluzione ideale;
- Fissato K ed N, al crescere di M logicamente aumentano i cache miss per ciascun algoritmo (ci sono più risorse distinte che competono tra di loro). Tuttavia è importante osservare come i tempi di esecuzione rimangano pressoché paragonabili, il che ha senso visto che nell'analisi delle complessità di ciascun algoritmo, il numero delle risorse distinte non rientrava tra i parametri che potevano influenzare i tempi di esecuzione;
- FIFO e LIFO in quasi tutti i test hanno presentato tempi di esecuzione minori rispetto ad Optimal, ma di contro registrano molto più cache miss. Ed è proprio qui che FIFO e LIFO nella realtà richiederebbero più tempo, dato che un cache miss corrisponde ad un cambio di contesto. La ricerca di un dato ai livelli inferiori di una certa memoria ha costi di elaborazione di ordini maggiori, e i tempi in questione sono quindi molto rilevanti. Tuttavia il progetto non ha considerato tali tempistiche, motivo per cui FIFO e LIFO sembrerebbero più performanti di Optimal, quantomeno a livello temporale;
- LIFO si presta ad essere il più veloce algoritmo tra quelli confrontati, se si osserva il tempo di esecuzione. E ciò è spiegato dal fatto che la politica LIFO implementata in questo sistema si riduce a continue sostituzioni nell'ultima posizione della cache occupata, dato che non si assume che le risorse abbiano

un intervallo di tempo in cui devono trovarsi nella cache, e di conseguenza possono rimanere in memoria per un tempo “infinito”. Dato che si opera sempre in una stessa cella, essa rimarrà costantemente caricata nei registri di alto livello, estremamente veloci. Aggiungendo il fatto che LIFO non preveda alcun riordinamento, l’esecuzione di tale algoritmo in questo contesto è estremamente veloce. Il vero contro, come già illustrato al punto precedente, risiede nell’elevato numero di cache miss che registra;

Le tabelle soprastanti, sono consultabili assieme ai singoli test al seguente link: https://drive.google.com/drive/folders/1iZTAOh8SKd2fA5NLHQAtQDp2tG9NZXip?usp=sharing

6 - Conclusioni e Sviluppi futuri

6.1 Conclusioni

Tramite l'analisi del problema si è potuto progettare, implementare e validare sperimentalmente un simulatore di politiche di caching per la gestione della memoria, mettendo a confronto strategie classiche (FIFO, LIFO) con una soluzione ottimale implementata sia tramite una logica intuitiva che tramite un algoritmo goloso.
Il cuore del progetto risiede nel superamento della soluzione Naive (ingenua, con con complessità $O(kn^2)$) tramite un'architettura ottimizzata. Integrando insieme un Albero Rosso-Nero, un Max-Heap e una HashMap di liste collegate (LinkedList) , si è ottenuto un algoritmo capace di operare con una complessità totale $O(n*\lg k)$.
I risultati sperimentali hanno confermato la solidità delle stime asintotiche, ma hanno anche sollevato un'importante riflessione. Sebbene la politica LIFO abbia registrato i tempi di calcolo CPU più bassi sulla JVM a causa della sua semplicità strutturale e della località della memoria hardware, il numero di cache miss registrati è decisamente più elevato rispetto a quelli di Optimal e Naive, con numeri paragonabili a quelli della politica FIFO.
In un sistema operativo, ogni cache miss genera un'interruzione che costringe la CPU a subire un cambio di contesto (context switch) e ad attendere i tempi di I/O della memoria secondaria. Di conseguenza, l'apparente velocità di esecuzione di LIFO nel simulatore viene totalmente annullata nella realtà hardware dall'overhead dei cambi di contesto (thrashing).

Di conseguenza, nonostante la soluzione Optimal richieda strutture di supporto e abbia tempi di elaborazioni più lunghi, resta comunque la migliore in quanto minimizza il numero di cache miss possibili, presentando un limite inferiore utile per il confronto con altre politiche di caching.

6.2 Sviluppi Futuri

Il simulatore sviluppato, grazie all'alto livello di modularità e disaccoppiamento del codice Java, si presta a essere la base per diverse estensioni future:

- Integrazione di Politiche Basate sulla Frequenza e Recenza (LRU e LFU): un naturale sviluppo consiste nell'implementazione degli algoritmi Least Recently Used (LRU) e Least Frequently Used (LFU). Tali politiche, pur non godendo della preveggenza matematica di Furthest-In-Future, tentano di approssimare

l'ottimo basandosi sullo storico reale degli accessi, offrendo un ulteriore terreno di confronto;

- Modellazione dei Costi di Dispositivo (Weighted Caching): Attualmente il simulatore assume che ogni risorsa abbia lo stesso costo di recupero. Un'estensione significativa potrebbe associare a ogni risorsa un "peso" in byte o un tempo di latenza differenziato (es. simulando il recupero da una rete locale rispetto a un server remoto), modificando le metriche di rimozione non solo sulla distanza temporale ma anche sul costo economico del miss;
- Supporto a Tracce di Input Dinamiche in Real-Time: evolvere il sistema affinché non analizzi solo file statici pre-elaborati, ma possa elaborare flussi di richieste in streaming in tempo reale, introducendo finestre mobili (sliding windows) per il pre-processing dinamico della sequenza;